Lecture 12: Semi-supervised learning, self-supervised learning

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 29, 2026

Announcement

- Last chance to discuss project topics in person: Today's TA office hour after class
- Reminder: Students will be asked to (optionally, for extra credit) provide feedback on each other's proposals. Feedback format:
  - Would you be able to summarize what the author(s) is/are proposing to do? If not, what is not clear?
  - Is the motivation clear and convincing? If not, why not?
  - Is the plan clear and feasible? If not, why not?
  - Any other comments or suggestions?

Semi-supervised learning
(Figure credits: Zhu ICML 2007 tutorial unless otherwise indicated)

Notation

(Borrowed from Jerry Zhu)

- Input x, label y
- Function to be learned $f : \mathcal{X} \longrightarrow \mathcal{Y}$
- Labeled data $(X_l, Y_l) = \{(x_{1:l}, y_{1:l})\}$
- Unlabeled data $X_u = \{x_{l+1:n}\}$
- Typically, $l \ll n$
- Test data $X_{test} = \{x_{n+1:}\}$

Techniques for combining labeled and unlabeled data

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data)

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data)
- Self-training (first model labeled data, then unlabeled data)

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)
- Generative models (similar to multi-task learning, but with density estimation losses)

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)
- Generative models (similar to multi-task learning, but with density estimation losses)
- Graph-based algorithms

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)
- Generative models (similar to multi-task learning, but with density estimation losses)
- Graph-based algorithms
- Multi-view algorithms

Techniques for combining labeled and unlabeled data

- Pre-training (first model unlabeled data, then labeled data)
- Self-training (first model labeled data, then unlabeled data)
- Multi-task learning (combine unlabeled loss with labeled loss)
- Generative models (similar to multi-task learning, but with density estimation losses)
- Graph-based algorithms
- Multi-view algorithms
- (You have used some of these already for the homeworks, and some of the projects are studying others)

Self-training

Self-training algorithm:

1. Train f from (X_l, Y_l)
2. Predict on $x \in X_u$
3. Add $(x, f(x))$ to labeled data
4. Repeat

- Typically, only $(x, f(x))$ pairs with high confidence are used

  Advantages

  - Very simple
  - A "wrapper" method: Can be applied with any existing model $f(\cdot)$
  - Has had some real-world successes (e.g. in speech and NLP)

Self-training

Self-training algorithm:

1. Train f from (X_l, Y_l)
2. Predict on $x \in X_u$
3. Add $(x, f(x))$ to labeled data
4. Repeat

- Typically, only $(x, f(x))$ pairs with high confidence are used

Advantages

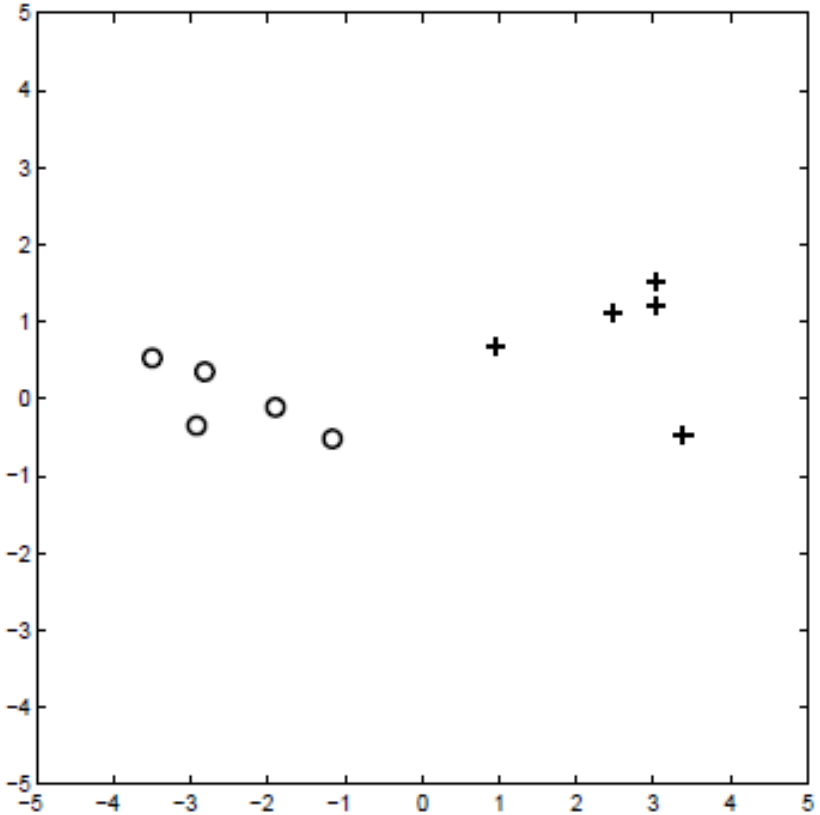
- Very simple
- A "wrapper" method: Can be applied with any existing model $f(\cdot)$
- Has had some real-world successes (e.g. in speech and NLP)

Disadvantages

- Very risky: Can easily diverge if some early mistakes are made
- Little theoretical understanding

Semi-supervised generative models

Consider learning a binary classifier from this labeled data set (X_l, Y_l):



Let's use a generative model with one Gaussian per class...

Semi-supervised generative models

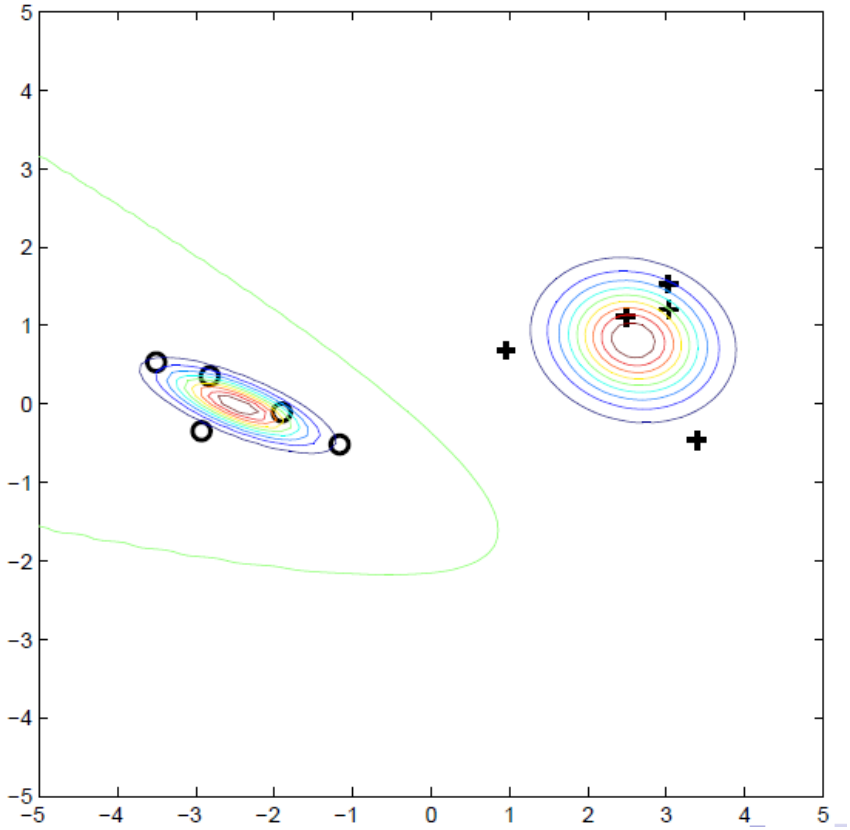
Let's use a generative model with one Gaussian per class...

$$\begin{aligned} p(x, y|\theta) &= p(y|\theta)p(x|y, \theta) \\ &= w_y \mathcal{N}(x; \mu_y, \Sigma_y) \end{aligned}$$

- Train by maximum likelihood (closed-form solution)
- Classify by finding the label with maximum probability: $p(y|x, \theta) = \frac{p(x,y|\theta)}{\sum_{y'} p(x,y'|\theta)}$

Semi-supervised generative models

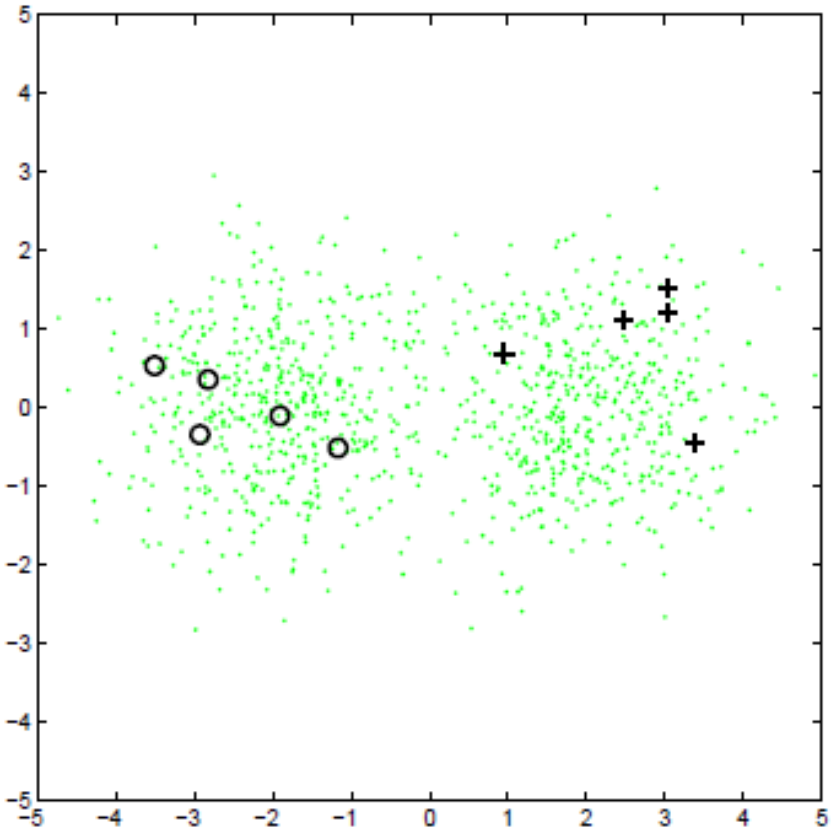
Here is the model learned from our labeled data:



Is this a good classifier?

Semi-supervised generative models

Now suppose we also have access to some unlabeled data:



Does this make us want to change our classifier?

Semi-supervised generative models

- Maximum likelihood estimation using labeled data only:

$$\log p(X_l, Y_l|\theta) = \sum_{i=1}^{l} \log p(y_i|\theta)p(x_i|y_i, \theta)$$

Semi-supervised generative models

- Maximum likelihood estimation using labeled data only:

$$\log p(X_l, Y_l|\theta) = \sum_{i=1}^{l} \log p(y_i|\theta)p(x_i|y_i, \theta)$$

- MLE using both labeled and unlabeled data:

$$\begin{aligned}\log p(X_l, Y_l, X_u|\theta) &= \sum_{i=1}^{l} \log p(y_i|\theta)p(x_i|y_i, \theta) \\ &+ \sum_{i=l+1}^{l+u} \log \left(\sum_{y} p(y|\theta)p(x_i|y, \theta) \right)\end{aligned}$$

Semi-supervised generative models

- Maximum likelihood estimation using labeled data only:

$$\log p(X_l, Y_l|\theta) = \sum_{i=1}^{l} \log p(y_i|\theta)p(x_i|y_i, \theta)$$

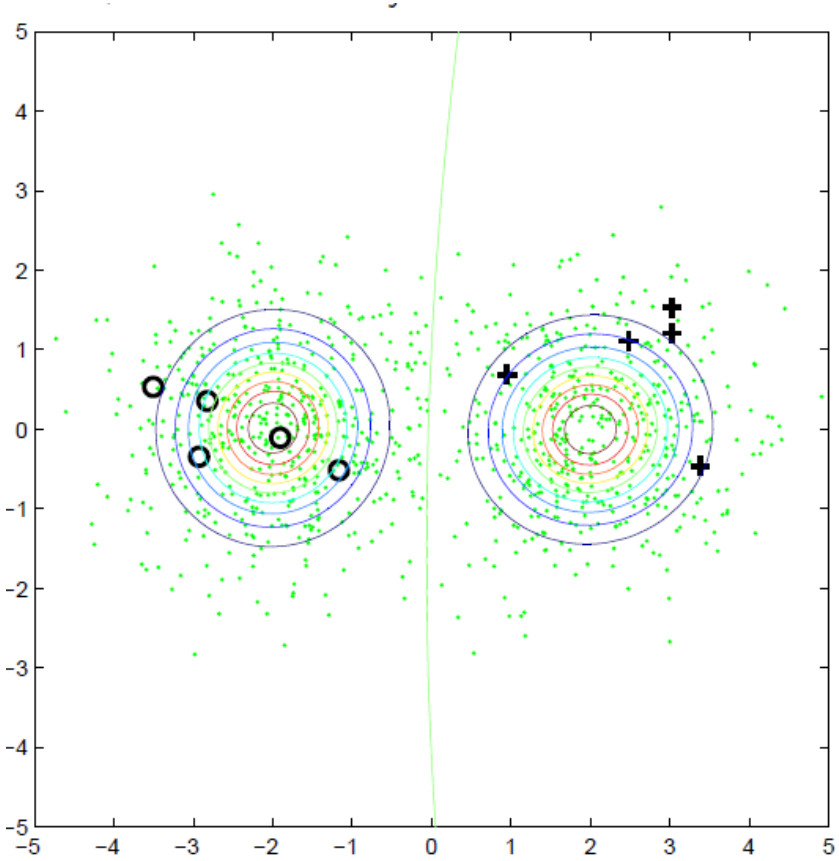
- MLE using both labeled and unlabeled data:

$$\begin{aligned}\log p(X_l, Y_l, X_u|\theta) &= \sum_{i=1}^{l} \log p(y_i|\theta)p(x_i|y_i, \theta) \\ &+ \sum_{i=l+1}^{l+u} \log \left(\sum_{y} p(y|\theta)p(x_i|y, \theta) \right)\end{aligned}$$

- Now we have hidden variables (labels of X_u) $\Longrightarrow$ use EM
- Same as EM for general Gaussian mixture learning, but some of the latent variables (component assignments) are hard labels while the rest are posteriors (as usual)

Semi-supervised generative models

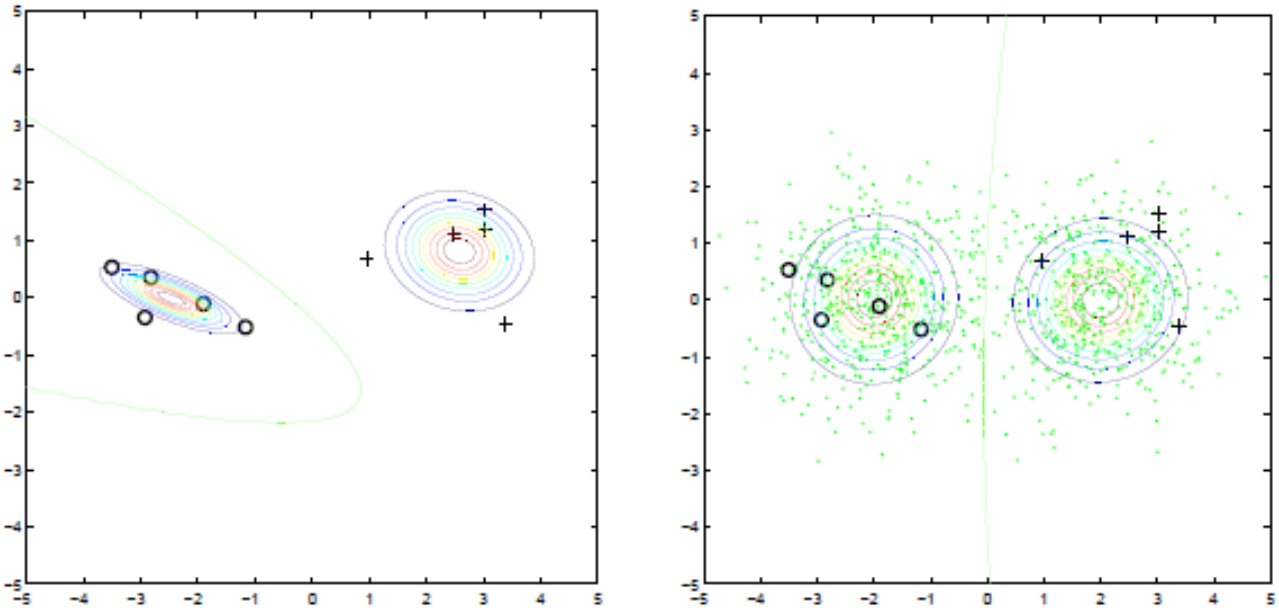
Maximum-likelihood model and classifier using both labeled and unlabeled data:



Semi-supervised generative models

Comparing supervised and semi-supervised classifiers side by side:

$p(X_l, Y_l|\theta)$ $p(X_l, Y_l, X_u|\theta)$



- Can be applied to other generative models, e.g. hidden Markov models (speech, text, video, other time series)
- Amounts to doing EM for the appropriate model, treating unseen labels as latent variables

Semi-supervised generative models

Inherits many of the qualities of generative models

- If the model is a good fit to the data, can be very effective
- Provides an intuitive, clear probabilistic interpretation
- But, doesn't work well if we don't have an accurate generative model for the unlabeled data
- And, sometimes we'd rather classifiy with a non-generative model

Cluster-then-label

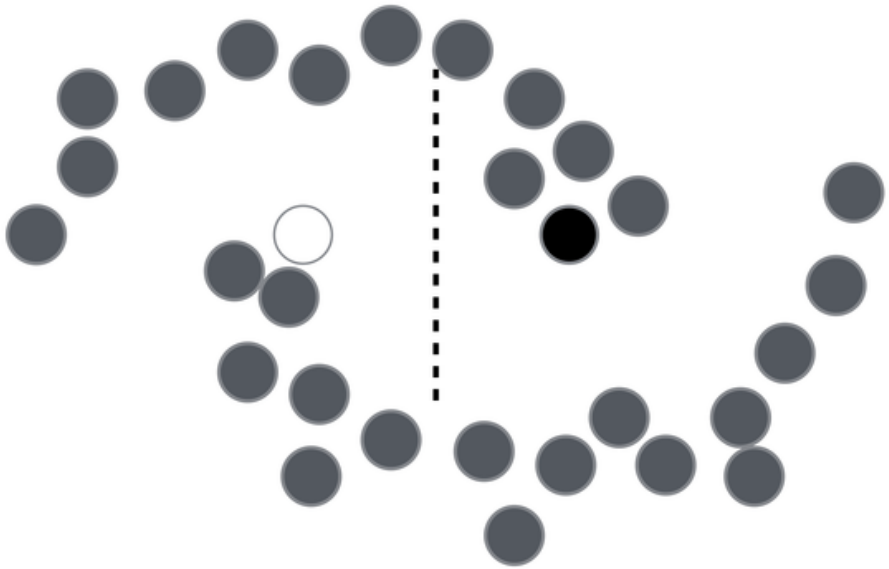
A popular cousin of generative models

- Cluster the data X_l, X_u
- Label all points in a cluster by the majority label of the labeled data in the cluster
- (Just like you are doing in HW3)

Makes fewer assumptions about the data distribution, but also more unstable

Graph-based algorithms [Zhu+ 2001-2003]

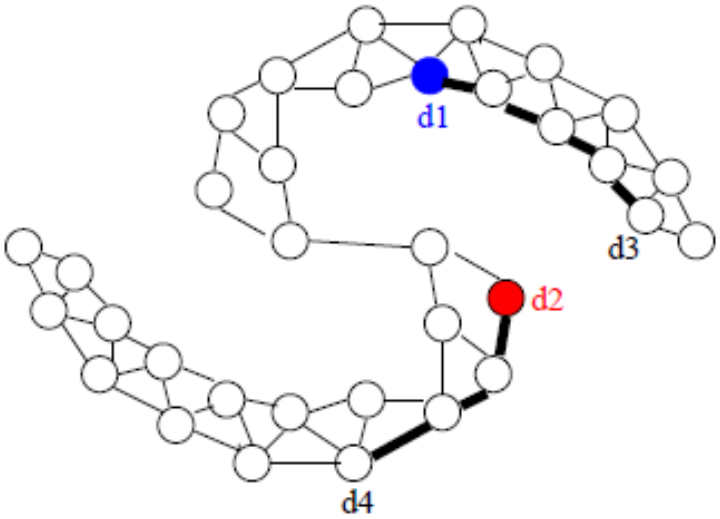
Some data sets don't fit simple generative models well, and clustering alone ignores labels



[Boney 2016]

Graph-based algorithms

We can take a hint from previous lectures on graph-based nonlinear dimensionality reduction:



- One node for each x in $X_l \cup X_u$
- Unweighted or weighted k-NN graph
- Or, fully connected graph with distance-based weights, e.g. $w_{ij} = e^{-||x_i - x_j||^2/\sigma^2}$
- Idea: We want $d3$ to be closer to $d1$ than to $d2$

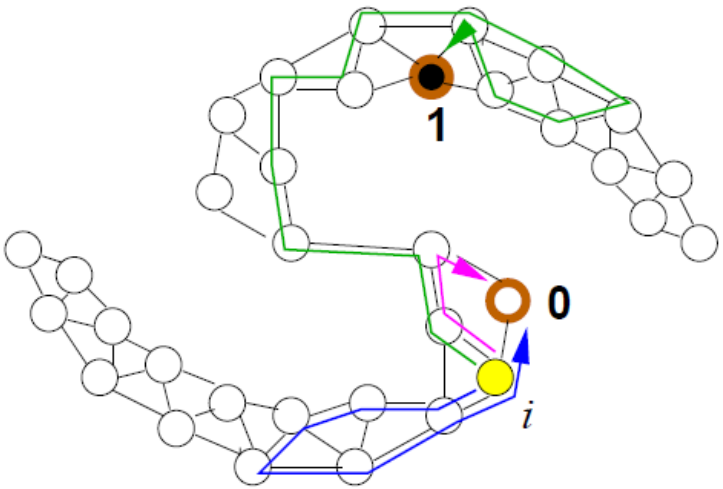
Graph-based algorithms

- We **could** use graph-based clustering approaches (e.g., mincut-based), but this ignores the labels
- Or, we could combine information from the labeled nodes with neighborhood preferences

Graph-based algorithms: Harmonic function

Assume binary classification for simplicity

- Consider the labels to be continuous-valued; then seek a function (the harmonic function) defined as follows:
  - Start from any node i and walk to j with probability $w_{ij} / \sum_k w_{ik}$
  - Stop when you hit a labeled node
  - The harmonic function is the probability $P(\text{hit label 1} | \text{start from } i)$
- Take this function to be the posterior probability of class 1 for each node



Graph-based algorithms: Harmonic function

One algorithm for computing the harmonic function:

- Set $f(x_i) = y_i$ for labeled nodes, initialize to 0 for unlabeled nodes
- Repeat until convergence: for all unlabeled nodes, $f(x_i) = \frac{\sum_{j \sim i} w_{ij} f(x_j)}{\sum_{j \sim i} w_{ij}}$
- I.e., at each iteration we take the average over the neighbors

Graph-based algorithms: Harmonic function

One algorithm for computing the harmonic function:

- Set $f(x_i) = y_i$ for labeled nodes, initialize to 0 for unlabeled nodes
- Repeat until convergence: for all unlabeled nodes, $f(x_i) = \frac{\sum_{j \sim i} w_{ij} f(x_j)}{\sum_{j \sim i} w_{ij}}$
- I.e., at each iteration we take the average over the neighbors

Can alternatively be computed via an eigenproblem on the graph Laplacian

- Similar to spectral clustering (but obeys labels)
- Also related to doing inference on a graphical model over the data labels

Graph-based algorithms: Harmonic function

One algorithm for computing the harmonic function:

- Set $f(x_i) = y_i$ for labeled nodes, initialize to 0 for unlabeled nodes
- Repeat until convergence: for all unlabeled nodes, $f(x_i) = \frac{\sum_{j \sim i} w_{ij} f(x_j)}{\sum_{j \sim i} w_{ij}}$
- I.e., at each iteration we take the average over the neighbors

Can alternatively be computed via an eigenproblem on the graph Laplacian

- Similar to spectral clustering (but obeys labels)
- Also related to doing inference on a graphical model over the data labels

One of a class of techniques referred to as "label propagation"

Graph-based algorithms

Some issues

- Some labels might be wrong, or we might be dealing with outliers
- New test points can't be directly handled (no "native" out-of-sample extension)

Graph-based algorithms

Some issues

- Some labels might be wrong, or we might be dealing with outliers
- New test points can't be directly handled (no "native" out-of-sample extension)

Possible solutions

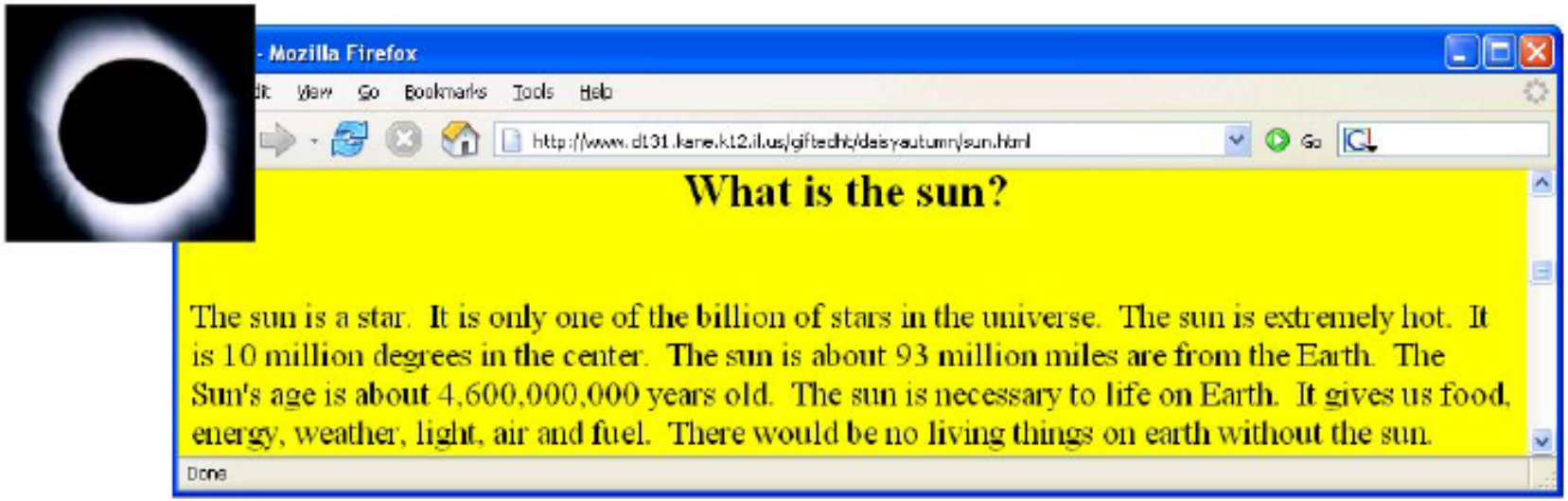
- Allow labeled points to be "relabeled" with some penalty
- Define a kernel that approximates the graph similarity
- Taken together, these extensions are referred to as "manifold regularization"

Multi-view algorithms

- We have already considered multi-view representation learning
- We now consider multi-view labeling (classification) in semi-supervised setting

Multi-view algorithms

Example: Image labeling using pixels + text





Multi-view algorithms: Co-training [Blum & Mitchell 1998]

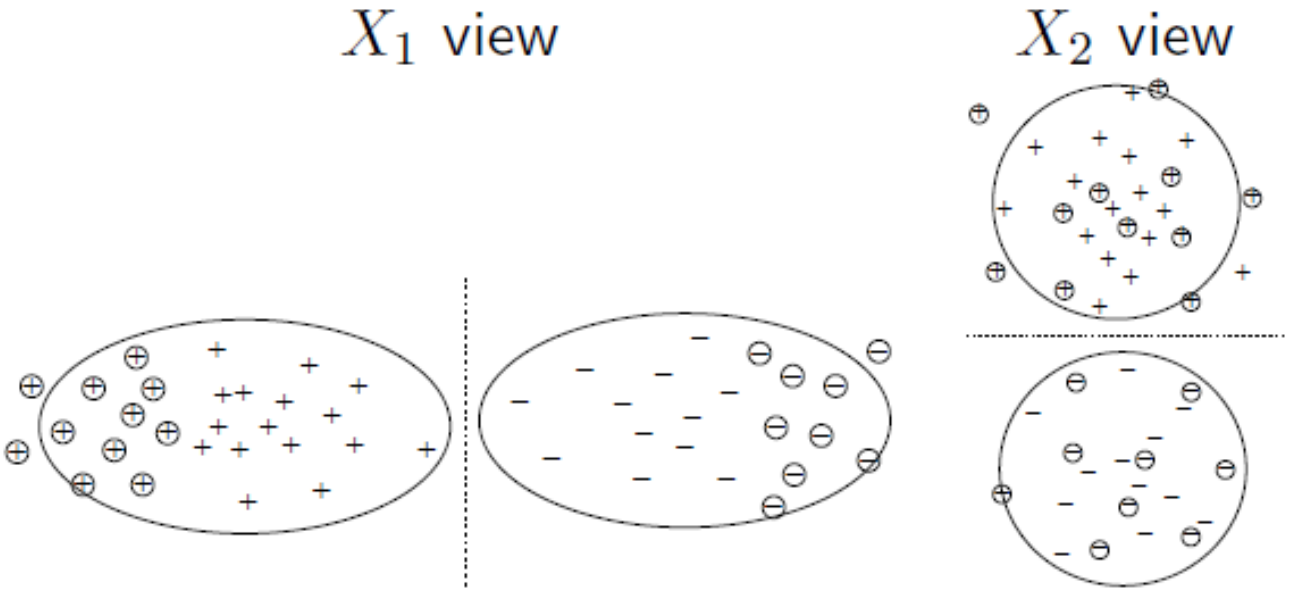
Setup:

- Feature vector can be naturally split into two views $x = [x^{(1)}\ x^{(2)}]$
- E.g., $x^{(1)}$ = image pixels, $x^{(2)}$ = text description
- Train two classifiers, one for each view
- Idea: The two classifiers "teach" each other; when one is unconfident, hopefully the other one is

Multi-view algorithms: Co-training

Assumptions

- $x^{(1)}$ or $x^{(2)}$ alone suffices for good classification, given enough data
- $x^{(1)}$ and $x^{(2)}$ are independent given the class label



Multi-view algorithms: Co-training

Algorithm [Blum & Mitchell 1998]

- Train two classifiers, $f^{(1)}$ from $(X_l^{(1)}, Y_l)$ and $f^{(2)}$ from $(X_l^{(2)}, Y_l)$
- Classify X_u with each $f^{(i)}$ separately
- Add the k most-confident $(x, f^{(1)}(x))$ pairs to $f^{(2)}$'s labeled data
- Add the k most-confident $(x, f^{(2)}(x))$ pairs to $f^{(1)}$'s labeled data
- Repeat

Multi-view algorithms: Co-training

Variants:

- Co-EM: Add all unlabeled data at each step, not just the top k, with some probability based on the confidence
- Create multiple random (fake) feature splits
- Generalized multi-view
  - No feature split at all
  - Just train multiple classifiers of different types
  - Classify the unlabeled data with all of the classifiers
  - Add majority vote label

Multi-view algorithms: Co-training

Advantages

- Wrapper method: Can be wrapped around any existing classifier and learning algorithm
- In the right circumstances, works very well

Disadvantages

- The right circumstances don't exist that often

Self-supervised representation learning

Self-supervised representation learning

Sometimes learners set up tasks for themselves to solve





Self-supervised representation learning

- Even if we only have unlabeled data, we may have some additional information that we can use to devise a supervised "proxy" or "pretext" task
- Examples of additional information: another modality, context

Self-supervised representation learning

- Even if we only have unlabeled data, we may have some additional information that we can use to devise a supervised "proxy" or "pretext" task
- Examples of additional information: another modality, context
- Even without any additional signal, we may be able to define pretext tasks from the input alone
- A good pretext task is one that, in order to solve it well, we need to represent the "useful" information in the input

Self-supervised representation learning

- Even if we only have unlabeled data, we may have some additional information that we can use to devise a supervised "proxy" or "pretext" task
- Examples of additional information: another modality, context
- Even without any additional signal, we may be able to define pretext tasks from the input alone
- A good pretext task is one that, in order to solve it well, we need to represent the "useful" information in the input
- **Terminology note**: This term is used fairly loosely, and the boundary between self-supervised and other types of unsupervised learning is often unclear

Self-supervised representation learning

How do we used the resulting model?

Self-supervised representation learning

How do we used the resulting model?

- Usually, it is a neural network of some kind
- Some layer of the network is considered the "learned representation" (e.g., middle layer for autoencoder)

Self-supervised representation learning

How do we used the resulting model?

- Usually, it is a neural network of some kind
- Some layer of the network is considered the "learned representation" (e.g., middle layer for autoencoder)
- Use the learned representation as input to your favorite supervised model for a downstream task (e.g., linear/nonlinear classifier)
- Optionally, update (fine-tune) parameters of learned representation model jointly with downstream model

Pretext tasks: Images

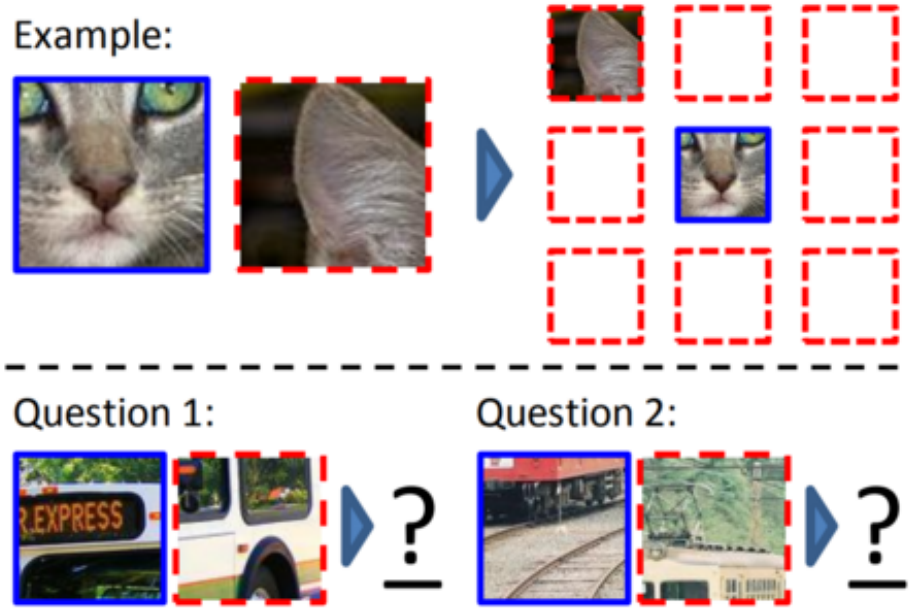
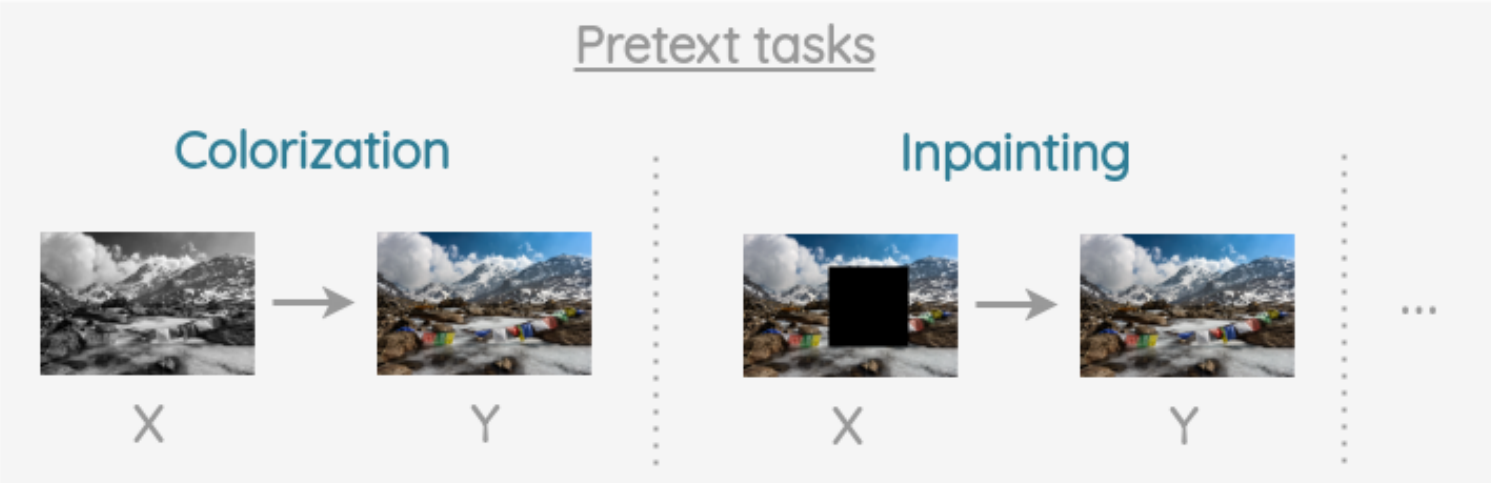


Figure 1. Our task for learning patch representations involves randomly sampling a patch (blue) and then one of eight possible neighbors (red). Can you guess the spatial configuration for the two pairs of patches? Note that the task is much easier once you have recognized the object!

Answer key: Q1: Bottom right Q2: Top center

[Doersch *et al.* 2015]

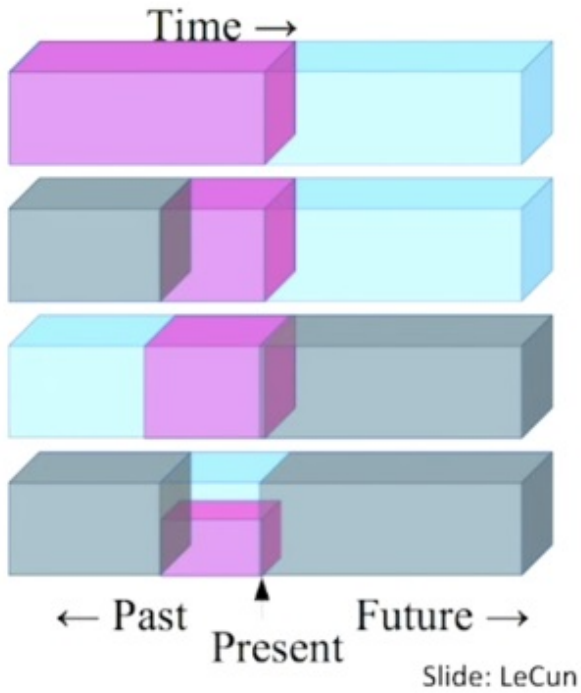
Pretext tasks: Images



[Chaudhary 2020]

Pretext tasks: Video

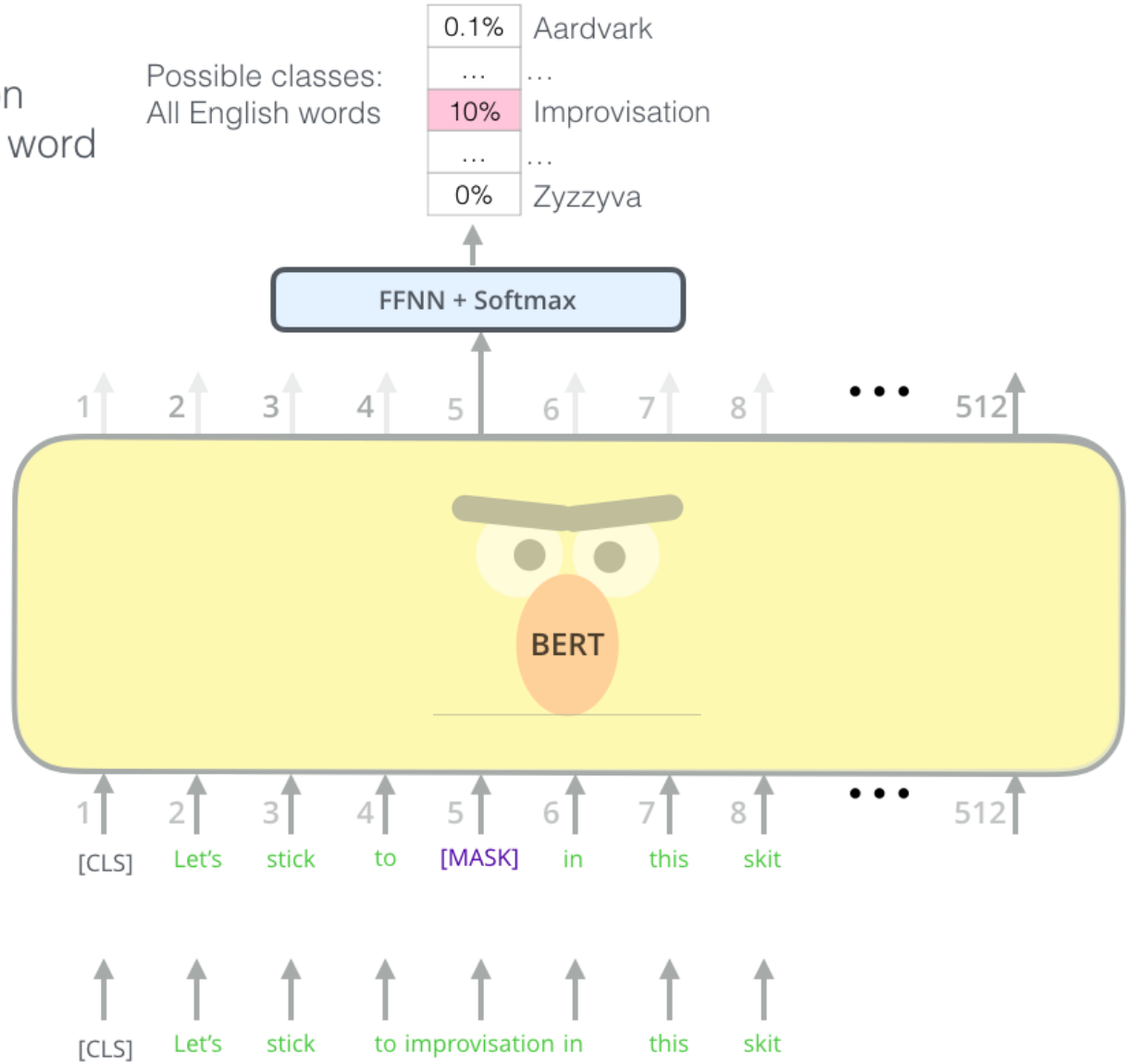
- Predict any part of the input from any other part.
- Predict the future from the past.
- Predict the future from the recent past.
- Predict the past from the present.
- Predict the top from the bottom.
- Predict the occluded from the visible
- Pretend there is a part of the input you don't know and predict that.



Note: Images can also be "arranged" into sequences, and sequence-based pretext tasks used

Pretext tasks: Text

Use the output of the masked word's position to predict the masked word



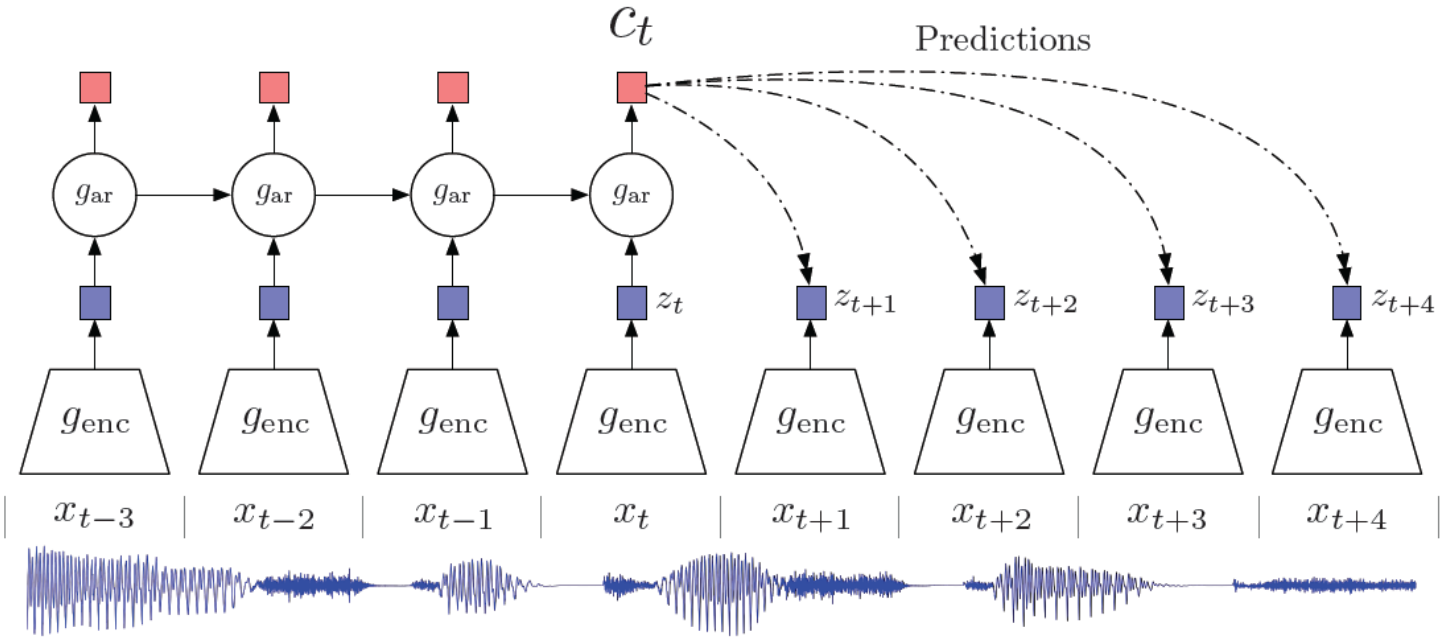
[Alammar 2018]

Tasks vs. losses

- For any given task, there may be multiple choices of losses
- E.g., for the task of "future prediction":
  - Predict the future value of the input?
  - Predict some feature of it?
  - Tell apart true vs. false futures?
- In practice, details of the loss can be important

Contrastive predictive coding (CPC)

Has been applied to speech, images, and other data types



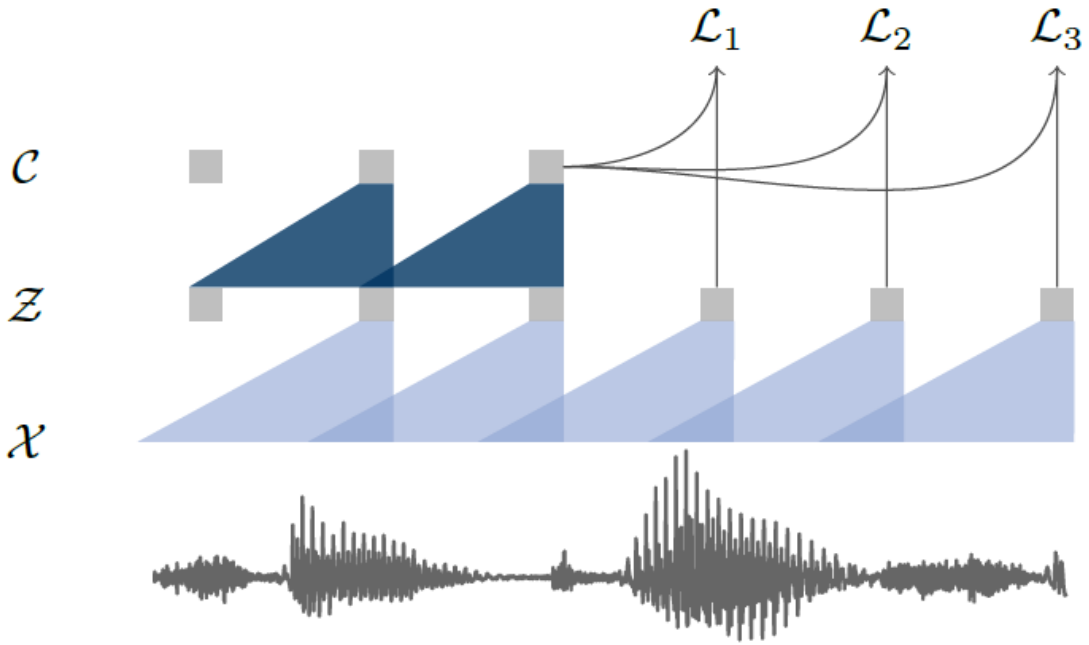
$$f_k(x_{t+k}, c_t) = \exp\left(z_{t+k}^T W_k c_t\right)$$

$$\mathcal{L}_{\text{N}} = -\mathop{\mathbb{E}}_{X}\left[\log \frac{f_k(x_{t+k}, c_t)}{\sum_{x_j \in X} f_k(x_j, c_t)}\right]$$

[van den Oord *et al.* 2018]

Contrastive predictive coding (CPC)

Wav2vec: A version of CPC that produced the first successful pre-training approach for automatic speech recognition

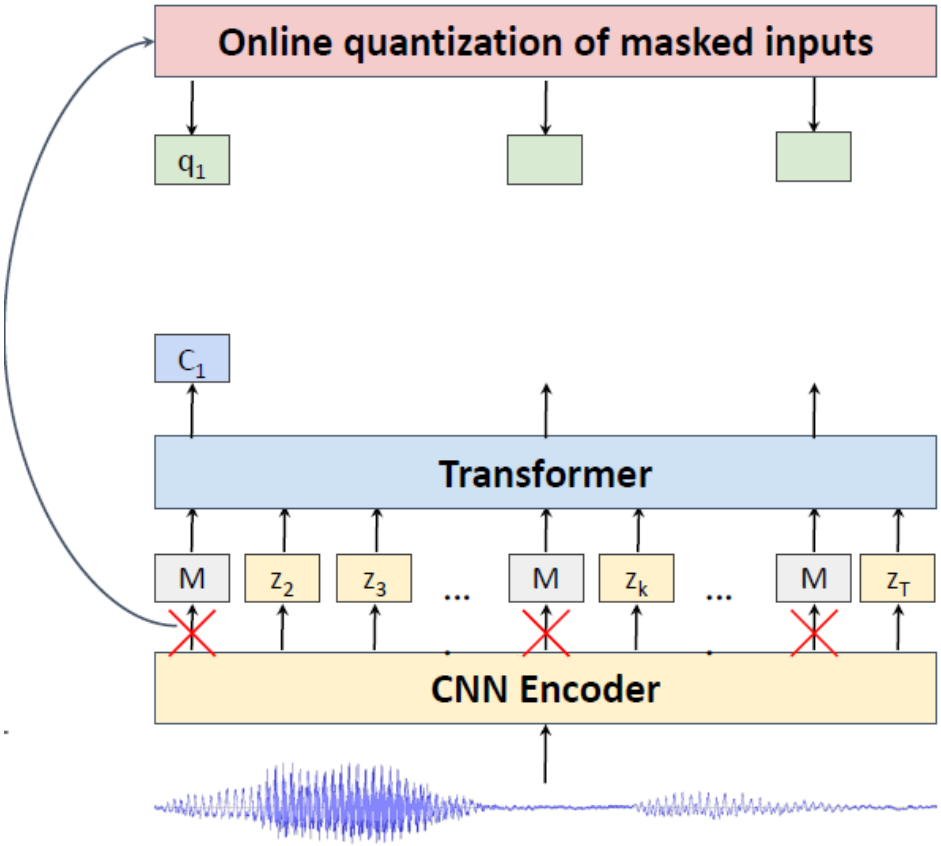


$$\mathcal{L}_k = -\sum_{i=1}^{T-k} \left(\log \sigma(\mathbf{z}_{i+k}^\top h_k(\mathbf{c}_i)) + \lambda \mathop{\mathbb{E}}_{\tilde{\mathbf{z}} \sim p_n} \left[\log \sigma(-\tilde{\mathbf{z}}^\top h_k(\mathbf{c}_i)) \right] \right)$$

[Schneider *et al.* 2019]

Combining contrastive loss and discretization

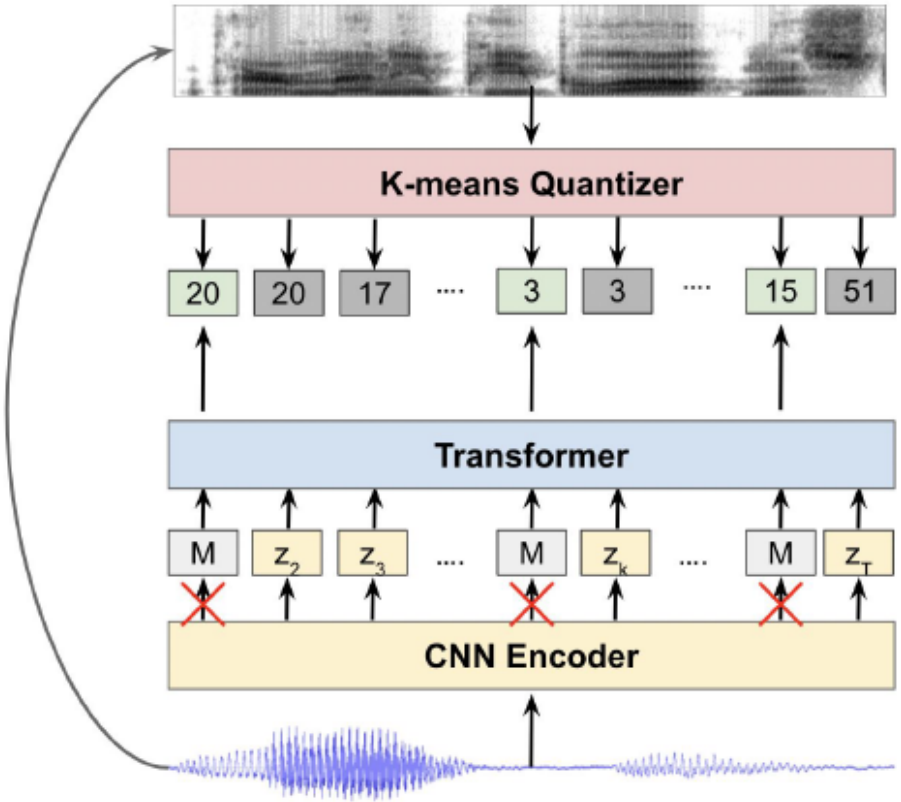
Wav2vec 2.0:



$$\mathcal{L}_m = -\log \frac{\exp(sim(\mathbf{c}_t, \mathbf{q}_t)/\kappa)}{\sum_{\tilde{\mathbf{q}} \sim \mathbf{Q}_t} \exp(sim(\mathbf{c}_t, \tilde{\mathbf{q}})/\kappa)}$$

HuBERT

HuBERT uses quantization like wav2vec 2.0, but uses a masked prediction loss $\sum_{t \in M} -\log p(y_t|X)$ and iterates quantization and re-training



[Hsu et al., "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," ICASSP 2021]